EFREI Paris

RAPPORT DE PROJET

Déploiement Automatisé d'une Infrastructure Cloud

Auteur	Hamza Mars
Date	12 mars 2026
GitHub	github.com/HamzaMrs/DevopsCloudComputing

1. Introduction

1.1 Contexte et Objectifs

Dans le cadre du cours d'Ingénierie du Cloud à l'EFREI Paris, ce projet consiste à déployer une infrastructure cloud complète de manière entièrement automatisée en utilisant Terraform, un outil d'Infrastructure as Code (IaC). Le scénario simulé est celui d'un ingénieur DevOps chargé de mettre en production une application web Flask sur Microsoft Azure.

L'Infrastructure as Code est aujourd'hui une pratique incontournable dans les équipes DevOps et SRE. Elle permet de décrire l'infrastructure sous forme de fichiers de configuration versionnables, reproductibles et auditables, en opposition aux déploiements manuels via des interfaces graphiques. Terraform, développé par HashiCorp, est l'un des outils les plus utilisés dans l'industrie pour cette approche.

1.2 Périmètre du Projet

Le projet couvre les éléments suivants :

- Déploiement automatisé d'une machine virtuelle Linux (Ubuntu 24.04) sur Azure
- Configuration d'un stockage cloud Azure Blob Storage pour les fichiers statiques (images, logs, fichiers applicatifs)
- Déploiement d'une base de données PostgreSQL managée (Azure Flexible Server)
- Développement d'une API REST Flask avec CRUD complet connectée au stockage et à la base de données
- Provisionnement automatique de la VM via cloud-init sans aucune intervention manuelle
- Tests complets de l'infrastructure déployée

1.3 Choix Technologiques

Microsoft Azure a été choisi comme provider cloud pour sa compatibilité avec Terraform via le provider officiel azurerm, et pour la disponibilité d'un abonnement Azure for Students. Le tableau suivant détaille l'ensemble des technologies utilisées :

Composant	Technologie	Version	Rôle
IaC	Terraform	v1.7.5	Déploiement et gestion de l'infrastructure
Cloud Provider	Microsoft Azure	France Central	Hébergement de l'ensemble des ressources
VM	Ubuntu Server LTS	24.04	Système d'exploitation de la machine virtuelle
Backend	Flask	3.0.0	Framework web Python pour l'API REST
Serveur WSGI	Gunicorn	21.2.0	Serveur de production pour Flask
Base de données	PostgreSQL	15 (Flexible Server)	Stockage des données persistantes
SDK Stockage	azure-storage-blob	12.19.0	Interaction avec Azure Blob Storage depuis Python
Gestionnaire de service	systemd	-	Démarrage automatique et supervision de Flask
Provisionnement VM	cloud-init	-	Configuration automatique au premier boot

2. Architecture du Projet

2.1 Structure des Fichiers

Le projet est organisé selon les bonnes pratiques DevOps, avec une séparation claire entre le code Terraform, l'application Flask et les scripts de provisionnement :

```
Cloud/
├── terraform/
│   ├── provider.tf              # Configuration du provider Azure (azurerm v3.117)
│   ├── variables.tf             # Déclaration des 10 variables d'entrée
│   ├── main.tf                  # 18 ressources Azure (VM, Storage, DB, réseau...)
│   ├── outputs.tf               # Sorties : IP publique, URL, FQDN PostgreSQL
│   ├── user_data.sh             # Script cloud-init (provisionnement automatique)
│   ├── terraform.tfvars.example # Exemple de variables (versionné)
│   └── terraform.tfvars         # Valeurs réelles (NON versionné - dans .gitignore)
├── app/
│   ├── app.py                   # Application Flask (375 lignes)
│   └── requirements.txt         # Dépendances Python
├── .gitignore                   # Exclut terraform.tfvars et fichiers sensibles
└── README.md                    # Documentation complète
```

2.2 Diagramme d'Architecture

L'architecture déployée sur Azure (France Central) est organisée autour d'un Virtual Network segmenté en deux sous-réseaux pour isoler la VM de la base de données :

```
Internet
    |
    | HTTP :5000 (NSG : ports 22, 80, 5000 ouverts)
    |
┌───┴──────────────────────────────────────────────────┐
│   Azure Cloud — France Central                       │
│                                                      │
│   Resource Group: cloud-efrei-rg                     │
│                                                      │
│   VNet: cloud-efrei-vnet (10.0.0.0/16)               │
│   │                                                  │
│   ├── Subnet Public (10.0.1.0/24)                    │
│   │   ├── IP Publique statique: 51.103.94.114        │
│   │   ├── NSG: cloud-efrei-nsg                       │
│   │   └── VM: cloud-efrei-vm                         │
│   │       ├── Ubuntu 24.04 LTS                       │
│   │       ├── Standard_B2ls_v2 (2 vCPU, 4 Go RAM)    │
│   │       └── Flask + Gunicorn (systemd)             │
│   │                                                  │
│   └── Subnet DB (10.0.2.0/24) — délégué PostgreSQL   │
│       ├── Zone DNS privée                            │
│       └── PostgreSQL Flexible Server v15             │
│           └── flaskdb (UTF-8)                        │
│                                                      │
│   Storage Account: cloudefreistrg2026hmz (LRS)       │
│   └── Container: staticfiles (accès privé)           │
│       ├── images/   ├── logs/   └── static/          │
└──────────────────────────────────────────────────────┘
```

2.3 Flux de Données

Les interactions entre les composants suivent le schéma suivant :

- L'utilisateur envoie des requêtes HTTP à l'API Flask via l'IP publique (51.103.94.114) sur le port 5000
- Le NSG (Network Security Group) filtre le trafic : seuls les ports 22 (SSH), 80 (HTTP) et 5000 (Flask) sont autorisés en entrée
- Flask traite les requêtes et interagit avec Azure Blob Storage via le SDK azure-storage-blob pour les opérations sur les fichiers
- Pour les opérations CRUD, Flask se connecte à PostgreSQL via le réseau privé du VNet (pas d'accès public à la base de données)
- La zone DNS privée permet à la VM de résoudre le nom du serveur PostgreSQL en interne

3. Étape 1 — Préparation de l'Environnement Terraform

3.1 Installation et Vérification de Terraform

Terraform v1.7.5 est installé sur macOS (darwin_arm64). La version est vérifiée avant de commencer :

```
$ terraform version
Terraform v1.7.5
on darwin_arm64
+ provider registry.terraform.io/hashicorp/azurerm v3.117.1
```

3.2 Connexion à Azure

La connexion à Azure se fait via Azure CLI. La commande az login ouvre un navigateur pour l'authentification, puis az account show permet de vérifier que le bon abonnement est actif :

```
$ az login
# Authentification via le navigateur...

$ az account show
{
  "name": "Azure for Students",
  "state": "Enabled",
  "id": "93de7bfe-4a9b-4153-aa5f-cd2226fd90fb"
}
```

3.3 Configuration du Provider Azure (provider.tf)

Le fichier provider.tf configure le provider azurerm et spécifie la version minimale de Terraform requise. L'option prevent_deletion_if_contains_resources = false est importante pour permettre à terraform destroy de supprimer le Resource Group même s'il contient encore des ressources :

```
terraform {
  required_version = ">= 1.0"
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.0"
    }
  }
}

provider "azurerm" {
  features {
    resource_group {
      prevent_deletion_if_contains_resources = false
    }
  }
}
```

3.4 Déclaration des Variables (variables.tf)

Le fichier variables.tf déclare 10 variables paramétrables qui rendent l'infrastructure dynamique et réutilisable dans n'importe quel environnement Azure. Les variables sensibles sont marquées sensitive = true pour éviter qu'elles apparaissent dans les logs Terraform :

Variable	Type	Description	Valeur configurée
project_name	string	Nom du projet — préfixe de toutes les ressources	cloud-efrei
location	string	Région Azure de déploiement	France Central
vnet_address_space	list(string)	Espace d'adressage du réseau virtuel	10.0.0.0/16
subnet_address_prefixes	list(string)	Préfixe du subnet principal (VM)	10.0.1.0/24
vm_size	string	Taille de la machine virtuelle	Standard_B2ls_v2
admin_username	string	Nom d'utilisateur administrateur de la VM	azureuser
admin_password	string (sensitive)	Mot de passe administrateur	— (terraform.tfvars)
storage_account_name	string	Nom unique du compte de stockage Azure	cloudefreistrg2026hmz
db_admin_username	string (sensitive)	Nom d'utilisateur PostgreSQL	flaskadmin
db_admin_password	string (sensitive)	Mot de passe PostgreSQL	— (terraform.tfvars)

Les valeurs sensibles sont stockées dans le fichier terraform.tfvars, qui est ajouté au .gitignore pour ne jamais être versionné sur GitHub. Un fichier terraform.tfvars.example (sans les vraies valeurs) est fourni à la place pour documenter la structure attendue.

3.5 Initialisation de Terraform

La commande terraform init télécharge le provider azurerm et initialise le répertoire de travail :

```
Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/azurerm from the dependency lock file
- Using previously-installed hashicorp/azurerm v3.117.1

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
hamzamars@Host-002 terraform %
```

4. Étape 2 — Déploiement de l'Infrastructure

4.1 Planification (terraform plan)

Avant tout déploiement, la commande terraform plan permet de prévisualiser exactement ce que Terraform va créer, modifier ou supprimer. C'est une bonne pratique à toujours exécuter avant un apply pour éviter les mauvaises surprises :

```
$ terraform plan

Terraform used the selected providers to generate the following execution plan.

Plan: 18 to add, 0 to change, 0 to destroy.
```

4.2 Déploiement (terraform apply)

Le déploiement complet est lancé avec terraform apply. Terraform liste toutes les ressources à créer, demande une confirmation, puis procède au déploiement. La durée totale est d'environ 12 minutes, principalement due au déploiement du PostgreSQL Flexible Server qui prend à lui seul 8 à 10 minutes :

```
$ terraform apply

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

azurerm_resource_group.main: Creating...
azurerm_resource_group.main: Creation complete after 2s
azurerm_virtual_network.main: Creating...
[...]
azurerm_linux_virtual_machine.main: Creation complete after 43s

Apply complete! Resources: 18 added, 0 changed, 0 destroyed.
```

4.3 Détail des 18 Ressources Créées

Terraform crée les 18 ressources Azure suivantes dans l'ordre correct, en respectant automatiquement les dépendances entre ressources (grâce aux références entre blocs et à depends_on) :

#	Type Terraform	Nom de la ressource	Description
1	azurerm_resource_group	cloud-efrei-rg	Conteneur logique de toutes les ressources Azure (France Central)
2	azurerm_virtual_network	cloud-efrei-vnet	Réseau virtuel principal (10.0.0.0/16)
3	azurerm_subnet	cloud-efrei-subnet	Sous-réseau pour la VM (10.0.1.0/24)
4	azurerm_subnet	cloud-efrei-db-subnet	Sous-réseau délégué pour PostgreSQL (10.0.2.0/24)
5	azurerm_network_security_group	cloud-efrei-nsg	Pare-feu : ports 22, 80, 5000 ouverts en entrée

#	Type Terraform	Nom de la ressource	Description
6	azurerm_public_ip	cloud-efrei-public-ip	Adresse IP publique statique (51.103.94.114, SKU Standard)
7	azurerm_network_interface	cloud-efrei-nic	Interface réseau de la VM (IP privée 10.0.1.4)
8	azurerm_network_interface_security_group_association	-	Association entre la NIC et le NSG
9	azurerm_storage_account	cloudefreistrg2026hmz	Compte de stockage Standard LRS, versioning activé
10	azurerm_storage_container	staticfiles	Conteneur Blob à accès privé
11	azurerm_storage_blob	images/.keep	Initialisation du dossier images/
12	azurerm_storage_blob	logs/.keep	Initialisation du dossier logs/
13	azurerm_storage_blob	static/.keep	Initialisation du dossier static/
14	azurerm_private_dns_zone	cloud-efrei.postgres.database.azure.com	Zone DNS privée pour PostgreSQL
15	azurerm_private_dns_zone_virtual_network_link	cloud-efrei-dns-link	Lien entre le VNet et la zone DNS privée
16	azurerm_postgresql_flexible_server	cloud-efrei-psql	PostgreSQL 15, B_Standard_B1ms, 32 Go, réseau privé
17	azurerm_postgresql_flexible_server_database	flaskdb	Base de données avec encodage UTF-8
18	azurerm_linux_virtual_machine	cloud-efrei-vm	VM Ubuntu 24.04 LTS, Standard_B2ls_v2 (2 vCPU, 4 Go)

4.4 Outputs Terraform

À la fin du déploiement, Terraform affiche les outputs définis dans outputs.tf. Ces informations permettent de se connecter à l'infrastructure sans avoir à consulter le portail Azure :

```
$ terraform output

app_url                  = "http://51.103.94.114:5000"
vm_public_ip             = "51.103.94.114"
storage_account_name     = "cloudefreistrg2026hmz"
storage_container_name   = "staticfiles"
storage_primary_endpoint = "https://cloudefreistrg2026hmz.blob.core.windows.net/"
postgresql_fqdn          = "cloud-efrei-psql.postgres.database.azure.com"
postgresql_server_name   = "cloud-efrei-psql"
resource_group_name      = "cloud-efrei-rg"
ssh_command              = "ssh azureuser@51.103.94.114"
```

4.5 Sécurité Réseau — Network Security Group

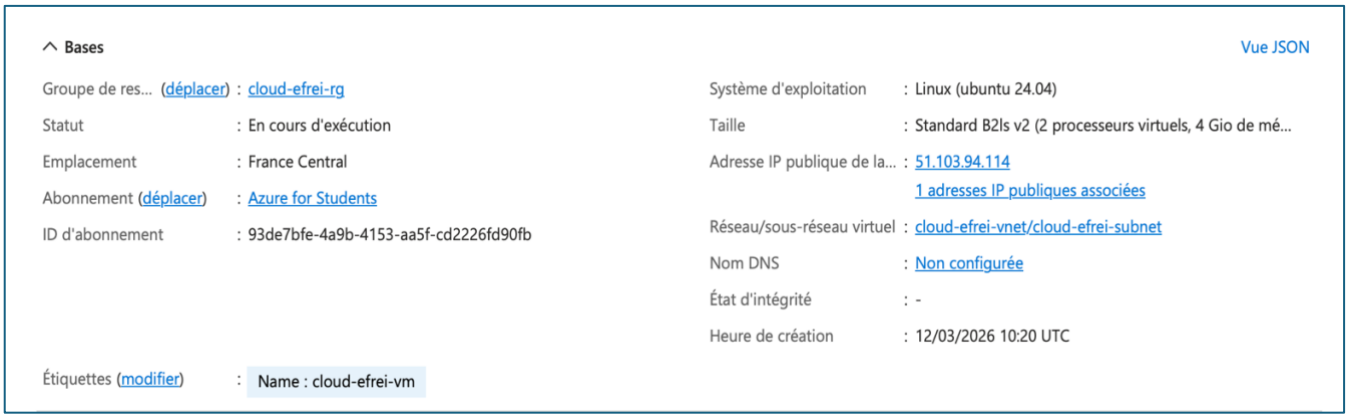
Le NSG (Network Security Group) agit comme un pare-feu au niveau de l'interface réseau de la VM. Trois règles de sécurité entrantes sont configurées :

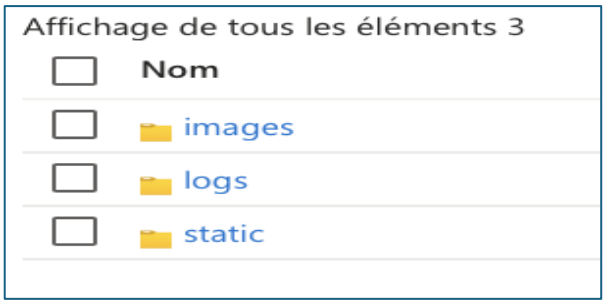
Priorité	Nom	Port	Protocole	Direction	Action	Justification
1001	SSH	22	TCP	Entrant	Autoriser	Administration de la VM
1002	Flask	5000	TCP	Entrant	Autoriser	Accès à l'API Flask
1003	HTTP	80	TCP	Entrant	Autoriser	Accès HTTP standard

Tout le trafic sortant est autorisé par défaut. Le trafic entrant non listé dans ces règles est refusé par la règle implicite DenyAllInBound d'Azure.

4.6 Captures d'écran — Ressources Azure déployées

Nom ↑	Type	Emplacement
cloud-efrei-nic	Interface réseau	France Central
cloud-efrei-nsg	Groupe de sécurité réseau	France Central
cloud-efrei-psql	Serveur flexible Azure Database pour PostgreSQL	France Central
cloud-efrei-public-ip	Adresse IP publique	France Central
cloud-efrei-vm	Machine virtuelle	France Central
cloud-efrei-vm_OsDisk_1_a0038e7e64a444d19e06867ec6e473ff	Disque	France Central
cloud-efrei-vnet	Réseau virtuel	France Central
cloud-efrei.postgres.database.azure.com	Zone DNS privé	Global
cloudefreistrg2026hmz	Compte de stockage	France Central





5. Étape 3 — Backend Flask et CRUD

5.1 Architecture de l'Application Flask

L'application Flask (app.py) est développée pour exposer une API REST complète. Elle se connecte à deux services Azure : Azure Blob Storage pour la gestion des fichiers, et PostgreSQL pour le stockage des données structurées. Au démarrage, elle initialise automatiquement la table items dans PostgreSQL si elle n'existe pas encore (CREATE TABLE IF NOT EXISTS).

5.2 Endpoints de l'API REST

Endpoints Système

Méthode	Route	Description	Retour
GET	/	Page d'accueil — liste tous les endpoints disponibles	JSON avec liste des routes
GET	/health	Health check complet : vérifie API, Blob Storage et PostgreSQL	{ api, database, storage }

Endpoints Fichiers — Azure Blob Storage

Méthode	Route	Description	Paramètres
GET	/files	Lister tous les fichiers du container staticfiles	-
POST	/files/upload	Uploader un fichier dans un dossier du container	file (multipart), folder (form)
GET	/files/<path:filename>	Télécharger un fichier par son chemin complet	filename dans l'URL
DELETE	/files/<path:filename>	Supprimer un fichier du container	filename dans l'URL

Endpoints CRUD — PostgreSQL

Méthode	Route	Description	Corps JSON
GET	/db/items	Lister tous les items triés par date de création	-
POST	/db/items	Créer un nouvel item en base	{ name, description, file_url }
GET	/db/items/<id>	Récupérer un item spécifique par son ID	-
PUT	/db/items/<id>	Mettre à jour les champs d'un item existant	{ name?, description?, file_url? }
DELETE	/db/items/<id>	Supprimer définitivement un item	-

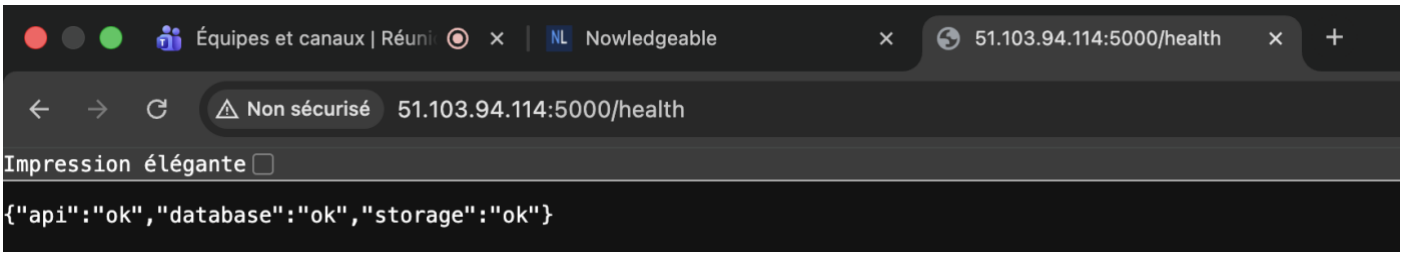
5.3 Tests des Endpoints API

Health Check — Vérification de l'état de l'infrastructure

Le premier test consiste à vérifier que les trois composants principaux (API, Blob Storage, PostgreSQL) sont opérationnels :

```
$ curl http://51.103.94.114:5000/health

{ "api": "ok", "database": "ok", "storage": "ok" }
```



Opérations sur les Fichiers (Azure Blob Storage)

Test d'upload d'un fichier dans le dossier logs/ du container staticfiles :

```
$ echo "fichier test EFREI" > test.txt

$ curl -X POST http://51.103.94.114:5000/files/upload \
       -F "file=@test.txt" \
       -F "folder=logs"

{
  "blob_name": "logs/test.txt",
  "container": "staticfiles",
  "message": "Fichier uploade avec succes"
}
```

Test de téléchargement et de suppression :

```
$ curl http://51.103.94.114:5000/files/logs/test.txt
fichier test EFREI

$ curl -X DELETE http://51.103.94.114:5000/files/logs/test.txt
{ "message": "Fichier 'logs/test.txt' supprime avec succes" }
```

Opérations CRUD sur la Base de Données PostgreSQL

Création d'un item en base :

```
$ curl -X POST http://51.103.94.114:5000/db/items \
       -H "Content-Type: application/json" \
       -d '{"name": "Logo EFREI", "description": "Logo officiel", "file_url":
"images/logo.png"}'

{
  "message": "Item cree avec succes",
  "item": {
    "id": 1,
    "name": "Logo EFREI",
    "description": "Logo officiel",
    "file_url": "images/logo.png",
    "created_at": "2026-03-12T10:55:29"
  }
}
```

Mise à jour d'un item :

```
$ curl -X PUT http://51.103.94.114:5000/db/items/1 \
       -H "Content-Type: application/json" \
       -d '{"name": "Logo EFREI (maj)", "description": "Nouveau logo 2026"}'
{ "message": "Item mis a jour avec succes", "item": { "id": 1, ... } }
```

Suppression d'un item :

```
$ curl -X DELETE http://51.103.94.114:5000/db/items/1
{ "message": "Item 1 supprime avec succes" }
```

6. Étape 4 — Automatisation du Déploiement

6.1 Principe du Provisionnement Automatique

L'un des objectifs centraux du projet est que la VM soit configurée automatiquement sans aucune intervention manuelle. Pour cela, Terraform utilise l'argument custom_data de la ressource azurerm_linux_virtual_machine pour injecter un script bash (user_data.sh) qui sera exécuté par cloud-init au premier démarrage de la machine.

Cette approche présente plusieurs avantages majeurs par rapport à une configuration manuelle via SSH :

- Reproductibilité : chaque nouvelle VM sera configurée de manière identique
- Traçabilité : le script est versionné dans Git avec le reste du code
- Intégration : les variables de configuration (clés, mots de passe, URLs) sont injectées dynamiquement par Terraform
- Zéro intervention : la VM est prête à servir des requêtes sans que l'opérateur ait besoin de s'y connecter

6.2 Injection Dynamique des Variables via templatefile()

Les credentials et paramètres de connexion sont injectés dynamiquement dans le script par Terraform au moment du terraform apply, via la fonction templatefile(). Ils ne sont jamais écrits en dur dans le code source :

```
custom_data = base64encode(templatefile("${path.module}/user_data.sh", {
  storage_account_name   = var.storage_account_name
  storage_account_key    = azurerm_storage_account.main.primary_access_key
  storage_container_name = var.storage_container_name
  db_host                = azurerm_postgresql_flexible_server.main.fqdn
  db_port                = "5432"
  db_name                = var.db_name
  db_username            = var.db_admin_username
  db_password            = var.db_admin_password
  app_port               = var.app_port
}))
```

6.3 Contenu du Script user_data.sh

Le script est exécuté en root au premier boot. Il comporte 6 étapes principales et redirige tous ses logs vers /var/log/user-data.log pour faciliter le débogage :

Étape	Actions réalisées
1 — Mise à jour système	apt-get update && apt-get upgrade -y
2 — Dépendances système	Installation de Python 3, pip, venv, postgresql-client, curl, git
3 — Environnement Python	Création du venv dans /opt/flask-app/venv, installation de Flask, Gunicorn, azure-storage-blob, psycopg2-binary
4 — Code de l'application	Écriture du fichier app.py avec les variables d'environnement injectées
5 — Service systemd	Création de /etc/systemd/system/flask-app.service avec Restart=always, 2 workers Gunicorn
6 — Initialisation DB	Connexion à PostgreSQL et CREATE TABLE IF NOT EXISTS items (...)

6.4 Configuration du Service systemd

L'application est gérée par systemd comme un service système, ce qui garantit son démarrage automatique après un reboot et sa relance en cas de crash :

```
[Unit]
Description=Flask Cloud EFREI Application (Azure)
After=network.target

[Service]
Type=simple
User=www-data
WorkingDirectory=/opt/flask-app
ExecStart=/opt/flask-app/venv/bin/gunicorn --bind 0.0.0.0:5000 --workers 2 --timeout 120
app:app
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

6.5 Vérification du Provisionnement

Après connexion SSH à la VM, on peut vérifier que le service Flask est actif et consulter les logs de provisionnement :

```
$ ssh azureuser@51.103.94.114

$ sudo systemctl status flask-app
● flask-app.service - Flask Cloud EFREI Application (Azure)
   Loaded: loaded (/etc/systemd/system/flask-app.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-03-12 10:23:19 UTC; 1h 16min ago
  Main PID: 10033 (gunicorn)
   Memory: 79.4M (peak: 81.1M)

$ sudo cat /var/log/user-data.log | tail -5
[OK] Service systemd cree et demarre.
[OK] Base de donnees initialisee.
==========================================
Provisionnement termine avec succes !
Application accessible sur le port 5000
```

```
● flask-app.service - Flask Cloud EFREI Application (Azure)
     Loaded: loaded (/etc/systemd/system/flask-app.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-03-12 10:23:19 UTC; 1h 16min ago
   Main PID: 10033 (gunicorn)
      Tasks: 3 (limit: 4675)
     Memory: 79.4M (peak: 81.1M)
        CPU: 1.417s
     CGroup: /system.slice/flask-app.service
             ├─10033 /opt/flask-app/venv/bin/python3 /opt/flask-app/venv/bin/gunicorn --bind 0.0.0.0:5000 --workers 2 --timeout 120 app:app
             ├─10035 /opt/flask-app/venv/bin/python3 /opt/flask-app/venv/bin/gunicorn --bind 0.0.0.0:5000 --workers 2 --timeout 120 app:app
             └─10036 /opt/flask-app/venv/bin/python3 /opt/flask-app/venv/bin/gunicorn --bind 0.0.0.0:5000 --workers 2 --timeout 120 app:app

Mar 12 10:23:19 cloud-efrei-vm systemd[1]: Started flask-app.service - Flask Cloud EFREI Application (Azure).
Mar 12 10:23:19 cloud-efrei-vm gunicorn[10033]: [2026-03-12 10:23:19 +0000] [10033] [INFO] Starting gunicorn 21.2.0
Mar 12 10:23:19 cloud-efrei-vm gunicorn[10033]: [2026-03-12 10:23:19 +0000] [10033] [INFO] Listening at: http://0.0.0.0:5000 (10033)
Mar 12 10:23:19 cloud-efrei-vm gunicorn[10033]: [2026-03-12 10:23:19 +0000] [10033] [INFO] Using worker: sync
Mar 12 10:23:19 cloud-efrei-vm gunicorn[10035]: [2026-03-12 10:23:19 +0000] [10035] [INFO] Booting worker with pid: 10035
Mar 12 10:23:19 cloud-efrei-vm gunicorn[10036]: [2026-03-12 10:23:19 +0000] [10036] [INFO] Booting worker with pid: 10036
```

7. Étape 5 — Tests et Destruction de l'Infrastructure

7.1 Plan de Tests

L'ensemble des fonctionnalités est testé selon le plan suivant, couvrant la couche infrastructure (Terraform), la couche réseau (accès à l'IP publique), la couche stockage (Azure Blob Storage) et la couche données (PostgreSQL) :

#	Catégorie	Test réalisé	Résultat
1	Infrastructure	terraform init — initialisation du provider Azure	OK
2	Infrastructure	terraform validate — validation syntaxique des fichiers .tf	OK
3	Infrastructure	terraform plan — 18 ressources planifiées correctement	OK
4	Infrastructure	terraform apply — déploiement des 18 ressources	OK
5	Infrastructure	terraform output — récupération de l'IP, URL, FQDN	OK
6	API	GET / — page d'accueil et liste des endpoints	OK
7	API	GET /health — API + Storage + Database tous à 'ok'	OK
8	Stockage	GET /files — liste des 3 dossiers du container	OK
9	Stockage	POST /files/upload — upload de test.txt dans logs/	OK
10	Stockage	GET /files/logs/test.txt — téléchargement et contenu correct	OK
11	Stockage	DELETE /files/logs/test.txt — suppression confirmée	OK
12	Base de données	POST /db/items — création d'un item (id=1)	OK
13	Base de données	GET /db/items — liste de tous les items	OK
14	Base de données	GET /db/items/1 — récupération par ID	OK
15	Base de données	PUT /db/items/1 — mise à jour des champs	OK
16	Base de données	DELETE /db/items/1 — suppression de l'item	OK
17	VM	SSH + systemctl status flask-app — service actif	OK

Résultat global : 17 tests sur 17 réussis — tous les objectifs du projet sont atteints.

7.2 Destruction de l'Infrastructure

Une fois les tests terminés, il est important de supprimer toutes les ressources Azure pour éviter de consommer inutilement les crédits de l'abonnement Azure for Students. La commande terraform destroy supprime les 18 ressources dans l'ordre inverse de leur création, en respectant les dépendances :

```
$ cd terraform/
$ terraform destroy

Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:
  - destroy

Plan: 0 to add, 0 to change, 18 to destroy.

Do you really want to destroy all resources?
  Only 'yes' will be accepted to confirm.

  Enter a value: yes

azurerm_storage_blob.images_folder: Destroying...
azurerm_linux_virtual_machine.main: Destroying...
[...]
azurerm_resource_group.main: Destruction complete after 2s

Destroy complete! Resources: 18 destroyed.
```

8. Problèmes Rencontrés et Solutions

Cette section documente les difficultés rencontrées au cours du projet et les solutions mises en place. Ces problèmes sont représentatifs des challenges habituels d'un projet DevOps en environnement cloud.

Problème 1 — Temps de déploiement du PostgreSQL Flexible Server

	Détail
Problème	Le déploiement du PostgreSQL Flexible Server prend 10 à 15 minutes, ce qui ralentit considérablement le cycle de développement. Pendant ce temps, si la VM est créée avant que PostgreSQL soit prêt, le script cloud-init échoue lors de l'initialisation de la base de données.
Impact	Échec du provisionnement de la VM, application non fonctionnelle au démarrage.
Solution	Utilisation de depends_on dans la ressource azurerm_linux_virtual_machine pour forcer Terraform à attendre que azurerm_postgresql_flexible_server_database soit entièrement créée avant de lancer la VM.
Résultat	Le déploiement est plus long mais fiable. La VM démarre toujours après que PostgreSQL soit prêt.

Problème 2 — Nom du Storage Account globalement unique

	Détail
Problème	Azure impose que le nom du Storage Account soit unique dans le monde entier, avec des contraintes strictes : 3 à 24 caractères, uniquement des minuscules et des chiffres. Lors des premières tentatives, les noms choisis étaient déjà pris.
Impact	Échec du terraform apply avec l'erreur : StorageAccountAlreadyTaken.
Solution	Utilisation d'un suffixe personnalisé avec les initiales et l'année (cloudefreistrg2026hmz). Le README documente cette contrainte et demande à chaque utilisateur de choisir un nom unique.
Résultat	Nom unique trouvé et déploiement réussi.

Problème 3 — SSL obligatoire pour PostgreSQL Flexible Server

	Détail
Problème	Le serveur PostgreSQL managé par Azure exige une connexion SSL. L'application Flask configurée avec sslmode=prefer échouait à se connecter avec l'erreur : SSL connection is required.
Impact	Le health check retournait database: error, et toutes les routes CRUD étaient en erreur 500.
Solution	Modification de la chaîne de connexion PostgreSQL dans le script user_data.sh pour utiliser sslmode=require.
Résultat	Connexion SSL établie avec succès, database: ok confirmé par le health check.

Problème 4 — Script cloud-init non répétable

	Détail
Problème	Le script user_data.sh est exécuté une seule fois au premier boot par cloud-init. Si une erreur se produit en cours d'exécution, le script s'arrête et la VM reste dans un état partiellement configuré. La seule solution est de détruire et recréer la VM.
Impact	Plusieurs cycles terraform destroy + terraform apply nécessaires lors du développement du script.
Solution	Ajout de set -e en début de script (arrêt immédiat en cas d'erreur), redirection de tous les logs vers /var/log/user-data.log, et configuration de systemd avec Restart=always pour que l'application redémarre automatiquement en cas de crash.
Résultat	Débogage facilité via les logs, et résilience de l'application en production.

Problème 5 — PostgreSQL sans accès réseau public

	Détail
Problème	Pour des raisons de sécurité, l'accès réseau public du PostgreSQL Flexible Server est désactivé (public_network_access_enabled = false). Il est donc impossible de s'y connecter directement depuis un poste local pour vérifier les données.
Impact	Pas de connexion directe via pgAdmin ou psql depuis la machine de développement.
Solution	Création d'un subnet dédié avec délégation Microsoft.DBforPostgreSQL/flexibleServers et d'une zone DNS privée. La VM communique avec PostgreSQL via le réseau interne du VNet. Pour les vérifications ponctuelles, connexion SSH à la VM puis psql depuis la VM.
Résultat	Architecture sécurisée : la base de données n'est accessible que depuis l'intérieur du VNet.

9. Conclusion

9.1 Bilan du Projet

Ce projet a permis de déployer une infrastructure cloud complète et fonctionnelle sur Microsoft Azure en utilisant exclusivement Terraform comme outil d'Infrastructure as Code. L'ensemble du processus — de la création du réseau virtuel au démarrage de l'application Flask — est automatisé et reproductible en une seule commande.

Les objectifs définis en introduction sont tous atteints : la machine virtuelle est opérationnelle, le stockage Blob est configuré et accessible via l'API, la base de données PostgreSQL est déployée en réseau privé, et l'application Flask expose un CRUD complet testé et validé.

9.2 Compétences Acquises

Domaine	Compétence acquise
Infrastructure as Code	Écriture et structuration de fichiers Terraform (provider, variables, main, outputs)
Cloud Azure	Déploiement de VM, Blob Storage, PostgreSQL Flexible Server, VNet, NSG, DNS privé
Sécurité cloud	Isolation réseau, NSG, accès privé à la base de données, secrets injectés via templatefile()
Automatisation	Provisionnement complet via cloud-init sans aucune intervention SSH manuelle
Backend Python	API REST Flask avec SDK azure-storage-blob, psycopg2, Gunicorn
Administration système	Gestion de services systemd, journalisation, supervision
DevOps	Versionnement Git, séparation des secrets, documentation technique (README, rapport)
FinOps	Choix de SKUs économiques, arrêt automatique, destruction des ressources après usage

9.3 Points Forts du Projet

- Une seule commande terraform apply suffit à créer l'intégralité de l'infrastructure (18 ressources Azure)
- L'application Flask est opérationnelle dès le démarrage de la VM, sans aucune connexion SSH manuelle
- Les secrets (clés d'accès, mots de passe) ne sont jamais écrits en dur dans le code — ils sont injectés dynamiquement par Terraform
- L'infrastructure est entièrement reproductible : elle peut être redéployée dans n'importe quelle région Azure en modifiant une seule variable
- La base de données est isolée dans un subnet privé, inaccessible depuis Internet
- 17 tests sur 17 réussis — l'ensemble du cahier des charges est couvert

9.4 Perspectives d'Amélioration

Plusieurs améliorations pourraient être apportées dans le cadre d'un projet de production :

- Mise en place d'un load balancer Azure pour la haute disponibilité
- Ajout d'un certificat SSL/TLS et d'un nom de domaine pour sécuriser les communications HTTPS
- Utilisation d'Azure Key Vault pour la gestion des secrets plutôt que terraform.tfvars
- Mise en place d'un pipeline CI/CD (GitHub Actions) pour automatiser les tests et le déploiement
- Configuration de Azure Monitor et Log Analytics pour la supervision en production